

CareerPath AI - Backend Java

Base de competências, evidências e diagnóstico | Documento KB-BE-001 | Versão 1.0

1. Objetivo e escopo

Este documento define a trilha oficial de competências para perfis de Backend Java usados pelo CareerPath AI. Ele foi estruturado para permitir recuperação semântica, diagnóstico de lacunas e geração de roadmap. Os níveis e pesos abaixo são parâmetros internos do projeto fictício e não representam exigências universais do mercado.

RAG_HINT: priorize chunks que contenham o skill_id da competência e o nível-alvo solicitado pelo usuário.

Para comparação de perfil, combine este documento com 04_niveis_competencia.pdf e 06_catalogo_vagas.csv.

2. Perfil-alvo

O profissional Backend Java constrói serviços, APIs e regras de negócio executadas no servidor. Em nível de estágio, espera-se base consistente em programação, POO, Git, banco relacional e HTTP; Spring Boot pode estar em aprendizado. Em nível júnior, espera-se autonomia para desenvolver pequenas funcionalidades com Spring Boot, persistência, testes e integração com banco.

3. Matriz resumida

ID Competência Estágio Júnior Prioridade Peso

BE-JAVA-01 Fundamentos de Java 2 3 Obrigatória 10

BE-POO-02 Programação Orientada a Objetos 2 3 Obrigatória 10

BE-GIT-03 Git e GitHub 2 3 Obrigatória 8

BE-SQL-04 SQL e modelagem relacional 2 3 Obrigatória 10

BE-REST-05 APIs REST e HTTP 2 3 Obrigatória 10

BE-SPRING-06 Spring Boot 1 3 Alta 10

BE-JPA-07 JPA/Hibernate 1 2 Alta 7

BE-TEST-08 Testes automatizados 1 2 Média 7

BE-SEC-09 Segurança básica de APIs 1 2 Média 6

BE-DOCKER-10 Docker básico 0 1 Diferencial 5

4. Competências detalhadas

BE-JAVA-01 - Fundamentos de Java

skill_id BE-JAVA-01

Categoria Linguagem

Nível mínimo para estágio 2

Nível mínimo para júnior 3

Prioridade Obrigatória

Peso sugerido 10

Definição: Capacidade de escrever e compreender programas Java, usando tipos, controle de fluxo, métodos, classes, coleções e tratamento de exceções.

Evidências aceitáveis: Projeto executável no GitHub; uso de ArrayList/Map; métodos bem separados; tratamento básico de exceções; README explicando execução.

Perguntas de diagnóstico: Consegue explicar diferença entre classe e objeto? Quando usaria List e Map? Como trataria uma entrada inválida?

Erros comuns: Decorar sintaxe sem compreender fluxo; colocar toda a lógica no método main; ignorar exceções; nomes pouco descritivos.

Próximo passo recomendado: Construir uma API ou aplicação de console com domínio real, persistência e testes básicos.

CHUNK_KEY: BE-JAVA-01 | **AREA:** BACKEND_JAVA | **TOPICS:** java, sintaxe, tipos, collections, exceptions

BE-POO-02 - Programação Orientada a Objetos

skill_id BE-POO-02

Categoria Fundamentos

Nível mínimo para estágio 2

Nível mínimo para júnior 3

Prioridade Obrigatória

Peso sugerido 10

Definição: Modelar domínio com classes coesas, encapsulamento, composição, interfaces e polimorfismo, evitando herança desnecessária.

Evidências aceitáveis: Projeto com entidades, serviços e interfaces; responsabilidades separadas; composição; testes demonstrando comportamento.

Perguntas de diagnóstico: Explique encapsulamento. Quando composição é melhor que herança? O que uma interface resolve?

Erros comuns: Classes gigantes; getters/setters para tudo; herança apenas para reaproveitar código; acoplamento entre camadas.

Próximo passo recomendado: Refatorar um projeto para separar domínio, serviço e infraestrutura.

CHUNK_KEY: BE-POO-02 | **AREA:** BACKEND_JAVA | **TOPICS:** poo, encapsulamento, herança, polimorfismo, interfaces

BE-GIT-03 - Git e GitHub

skill_id BE-GIT-03

Categoria Ferramentas

Nível mínimo para estágio 2

Nível mínimo para júnior 3

Prioridade Obrigatória

Peso sugerido 8

Definição: Usar controle de versão de forma segura: commits pequenos, branches,

merge/rebase em nível básico, resolução de conflitos e colaboração por pull request.

Evidências aceitáveis: Histórico de commits coerente; README; branches ou PRs; .gitignore correto.

Perguntas de diagnóstico: Qual a diferença entre commit e push? Como desfaria uma alteração? Como resolveria um conflito?

Erros comuns: Commits gigantes; mensagens vagas; subir segredos; versionar arquivos de build.

Próximo passo recomendado: Trabalhar com feature branches e abrir PRs descritivos.

CHUNK_KEY: BE-GIT-03 | **AREA:** BACKEND_JAVA | **TOPICS:** git, github, branch, commit, pull request

BE-SQL-04 - SQL e modelagem relacional

skill_id BE-SQL-04

Categoria Banco de Dados

Nível mínimo para estágio 2

Nível mínimo para júnior 3

Prioridade Obrigatória

Peso sugerido 10

Definição: Consultar e modelar bancos relacionais usando SELECT, JOIN, agregações, chaves, constraints, índices básicos e transações.

Evidências aceitáveis: Schema SQL; consultas com JOIN; migrations; aplicação persistindo dados.

Perguntas de diagnóstico: Quando usar INNER JOIN? O que é chave estrangeira? Por que um índice pode ajudar ou prejudicar?

Erros comuns: SELECT * em tudo; ausência de constraints; duplicação de dados; concatenar SQL inseguro.

Próximo passo recomendado: Criar banco PostgreSQL para aplicação e escrever consultas reais.

CHUNK_KEY: BE-SQL-04 | **AREA:** BACKEND_JAVA | **TOPICS:** sql, joins, constraints, normalização, postgresql

BE-REST-05 - APIs REST e HTTP

skill_id BE-REST-05

Categoria Web

Nível mínimo para estágio 2

Nível mínimo para júnior 3

Prioridade Obrigatória

Peso sugerido 10

Definição: Projetar endpoints HTTP coerentes, usar verbos, status codes, JSON, validação, paginação e tratamento de erros.

Evidências aceitáveis: API com CRUD, validações, respostas 2xx/4xx/5xx adequadas e

documentação.

Perguntas de diagnóstico: Diferença entre POST e PUT? Quando retornar 404? O que é idempotência?

Erros comuns: Sempre retornar 200; endpoints verbosos; lógica de negócio no controller; mensagens de erro inconsistentes.

Próximo passo recomendado: Expor CRUD REST documentado e testar via Postman/curl.

CHUNK_KEY: BE-REST-05 | **AREA:** BACKEND_JAVA | **TOPICS:** rest, http, json, status code, endpoints

BE-SPRING-06 - Spring Boot

skill_id BE-SPRING-06

Categoria Framework

Nível mínimo para estágio 1

Nível mínimo para júnior 3

Prioridade Alta

Peso sugerido 10

Definição: Construir aplicações Java com Spring Boot usando injeção de dependência, controllers, services, repositories, configuração e validação.

Evidências aceitáveis: Projeto Spring Boot com arquitetura em camadas, configuração por ambiente e endpoints funcionais.

Perguntas de diagnóstico: O que é injeção de dependência? Qual responsabilidade de Controller, Service e Repository?

Erros comuns: Field injection; controller com regra de negócio; configuração fixa no código; ausência de DTOs.

Próximo passo recomendado: Criar API Spring Boot + PostgreSQL + validação.

CHUNK_KEY: BE-SPRING-06 | **AREA:** BACKEND_JAVA | **TOPICS:** spring boot, dependency injection, controller, service,

repository

BE-JPA-07 - JPA/Hibernate

skill_id BE-JPA-07

Categoria Persistência

Nível mínimo para estágio 1

Nível mínimo para júnior 2

Prioridade Alta

Peso sugerido 7

Definição: Mapear entidades relacionais, relacionamentos, transações e consultas, compreendendo lazy/eager e riscos de N+1.

Evidências aceitáveis: Entidades com relacionamentos; repositories; migrations; tratamento de transações.

Perguntas de diagnóstico: O que é ORM? Qual diferença entre lazy e eager? O que é problema

N+1?

Erros comuns: Expor entidade diretamente; cascades sem entender; relacionamentos bidirecionais excessivos.

Próximo passo recomendado: Persistir domínio com JPA e observar consultas SQL geradas.

CHUNK_KEY: BE-JPA-07 | **AREA:** BACKEND_JAVA | **TOPICS:** jpa, hibernate, orm, entities, relationships

BE-TEST-08 - Testes automatizados

skill_id BE-TEST-08

Categoria Qualidade

Nível mínimo para estágio 1

Nível mínimo para júnior 2

Prioridade Média

Peso sugerido 7

Definição: Criar testes unitários e de integração para regras críticas e endpoints, usando JUnit e mocks apenas quando necessários.

Evidências aceitáveis: Testes executáveis no repositório; casos felizes e erros; cobertura de regras importantes.

Perguntas de diagnóstico: Diferença entre unitário e integração? Quando usar mock? O que torna um teste frágil?

Erros comuns: Testar implementação em vez de comportamento; excesso de mocks; testes sem asserts úteis.

Próximo passo recomendado: Adicionar testes ao service e ao fluxo HTTP principal.

CHUNK_KEY: BE-TEST-08 | **AREA:** BACKEND_JAVA | **TOPICS:** junit, mockito, unit tests, integration tests

BE-SEC-09 - Segurança básica de APIs

skill_id BE-SEC-09

Categoria Segurança

Nível mínimo para estágio 1

Nível mínimo para júnior 2

Prioridade Média

Peso sugerido 6

Definição: Entender autenticação, autorização, hashing de senha, segredos, CORS e validação de entrada em APIs.

Evidências aceitáveis: Autenticação simples; senhas com hash; secrets por variável de ambiente; autorização por papel.

Perguntas de diagnóstico: Por que não guardar senha em texto puro? Diferença entre autenticação e autorização?

Erros comuns: JWT sem expiração; secret no Git; confiar em dados do cliente.

Próximo passo recomendado: Implementar login seguro e autorização simples em projeto de

estudo.

CHUNK_KEY: BE-SEC-09 | AREA: BACKEND_JAVA | TOPICS: authentication, authorization, jwt, password hashing, secrets

BE-DOCKER-10 - Docker básico

skill_id BE-DOCKER-10

Categoria Deploy

Nível mínimo para estágio 0

Nível mínimo para júnior 1

Prioridade Diferencial

Peso sugerido 5

Definição: Empacotar aplicação e dependências em containers e executar ambiente reproduzível localmente.

Evidências aceitáveis: Dockerfile válido; docker-compose com app e banco; instruções no README.

Perguntas de diagnóstico: Diferença entre imagem e container? Para que serve Docker Compose?

Erros comuns: Imagem enorme; secrets no Dockerfile; ausência de healthcheck.

Próximo passo recomendado: Containerizar API e banco.

CHUNK_KEY: BE-DOCKER-10 | AREA: BACKEND_JAVA | TOPICS: docker, container, dockerfile, compose

5. Projetos de evidência recomendados

Projeto A - API de Biblioteca: CRUD de livros, usuários e empréstimos; Spring Boot; PostgreSQL; validação; tratamento de erros; testes; Docker opcional. Evidencia BE-JAVA-01, BE-POO-02, BE-SQL-04, BE-REST-05 e BE-SPRING-06.

Projeto B - Sistema de Academia: alunos, planos, matrículas e pagamentos fictícios; regras de validade; persistência; relatórios simples. Evidencia modelagem de domínio e regras de negócio.

Projeto C - Help Desk: abertura e acompanhamento de chamados, prioridade, status e histórico. Pode incluir autenticação e autorização.

6. Perguntas que o agente deve conseguir responder

Exemplos: "Sei Java, Git e SQL. Estou pronto para estágio backend?"; "O que falta para uma vaga que pede Spring Boot?"; "Qual projeto comprova melhor REST e JPA?"; "Devo estudar Docker antes de Spring?".

7. Regras específicas de recomendação

Para Backend Java, lacunas em Java, POO, Git, SQL e HTTP devem aparecer antes de diferenciais. Spring Boot deve subir para prioridade máxima quando a vaga o marcar como obrigatório. Docker nunca deve compensar ausência de fundamentos de Java. Testes ganham prioridade quando o perfil já possui Spring e persistência.